CampusExchange — University Room Exchange Platform

Master Project Documentation, Architecture Specification & Zero-Knowledge System Guide

Target Audience: Students, Wardens, Deans, Developers	**Tech Stack:** Python 5.x / Django / SQLite (ACID) / NetworkX
Core Invariant: Zero Partial Room Transfers (Row Locks)	**Matching Engine:** Multi-Hop Cyclic Graph Traversal (DFS 2-5)

1. Executive Summary & The Problem It Solves

What is this project?
In university campuses, hundreds of students are allocated hostel rooms each semester. Frequently, roommates find themselves mismatched, situated far from classes, on an inconvenient floor, or wishing to live next to classmates. Traditionally, room changes require manual applications, bureaucratic paperwork, and chaotic student negotiation.

The Fundamental Challenge: Direct Swaps Are Rare!
If *Student A* wants *Student B's* room, it is very unlikely that *Student B* also wants *Student A's* exact room. Instead, room preferences form **multi-party chains**:
• **Student A** wants Room 203 (occupied by B)
• **Student B** wants Room 304 (occupied by C)
• **Student C** wants Room 412 (occupied by D)
• **Student D** wants Room 101 (occupied by A)
If all four students agree and the Warden approves, **everyone shifts one step forward simultaneously** and everyone gets their dream room!

CampusExchange automates this entire lifecycle: discovering multi-party cycles using graph theory algorithms, collecting unanimous student consensus, giving wardens a one-click atomic approval queue, and issuing verifiable QR-coded transfer passes.

2. The End-to-End Operational Lifecycle

A room change travels through 6 distinct, tamper-proof stages. No step can be bypassed.

Phase 1: Student Registration & Inventory Allotment

A student creates an account or logs in. The system checks their university enrollment ID, gender, and current bed allocation (e.g., Tagore Block A, Room 101, Bed 1).

Phase 2: Submitting an Exchange Request

The student specifies what they want: preferred hostel (e.g., Sarojini), preferred room type (Single, Double), floor preferences, or a specific desired room. Before accepting the request, the platform verifies fee clearances, absence of disciplinary holds, and cooldown intervals.

Phase 3: Bounded Cycle Discovery (The Math Engine)

The system builds a directed graph of all students and their desired rooms. It automatically uncovers 2-party direct swaps, 3-party triangles, 4-party loops, and 5-party circular rotations.

Phase 4: Proposal Initiation & 100% Student Consensus

When a student selects a discovered cycle, an official Proposal is drafted. Every student in the cycle receives a notification. All participants must vote 'Accept'. If even one person declines or the proposal times out, it is cleanly canceled.

Phase 5: Warden Review & Atomic Execution

Once 100% consensus is achieved, the proposal reaches the Hostel Warden's queue. The Warden inspects student profiles, disciplinary notes, and room moves. With one click, the system acquires row-level database locks, atomically swaps all beds, and logs immutable audit trails.

Phase 6: Verifiable Transfer Pass & Key Handover

Students receive an official Digital Transfer Pass with a cryptographic QR code. They present this pass to the Hostel Caretaker to return their old keys and collect keys to their new room.

3. Complete Project Architecture & Directory Structure

CampusExchange is structured cleanly according to professional Django architectural best practices, dividing concerns into modular applications, specialized services, and isolated presentation layers.

Directory / File	Classification	Function & How It Works
`manage.py`	CLI Tool	Django command-line utility for database migrations, tests, and custom seed commands.
`edurev_swap/`	Project Core	Global configuration directory containing settings.py (installed apps, DB, auth backends), urls.py (root URL routing), and wsgi.py.
`accounts/`	Django App	Handles authentication, user identities, roles (Student, Warden, Chief Warden, Admin, Dean), and the instant Demo Persona Switcher.
`hostel/`	Django App	Manages campus physical assets: Hostels, Blocks, Floors, Rooms, and Beds. Tracks which student is currently assigned to which bed.
`exchange/`	Django App	Core engine orchestrating exchange requests, proposals, voting consensus, approvals, and official transfer passes.
`exchange/services/`	Business Logic	Modular services layer decoupling business rules from HTTP views (matching engine, occupancy locks, eligibility, audit).
`templates/`	UI Layer	Jinja2/Django HTML templates including student cockpit, warden approvals, cycle explorer, and transfer certificates.
`static/`	Frontend Assets	CSS stylesheets, interactive JS graph rendering scripts, and campus visual media.
`tests/`	QA / Verification	Automated test suite, including multi-threaded race-condition and concurrency safety tests.

4. Deep Dive: What Every Component Does

A. The accounts Application (User Management & RBAC)

• **Custom User Model (models.py):** Extends Django's base user to include University Roll Number, Gender (crucial for hostel segregation), Academic Year/Batch, Disciplinary Status, and Fee Clearance flags.
• **Universal Auth Backend (backends.py):** Allows students and faculty to log in seamlessly with either Username, Email address, or University ID.
• **Role-Based Access Control (middleware.py & context_processors.py):** Dynamically regulates endpoint access. Students cannot view warden approvals; wardens cannot alter admin policies. Exposes an instant 1-click **Demo Persona Switcher** in the top navbar.
• **Management Seeder (management/commands/seed_demo.py):** Pre-configures realistic accounts for Aarav, Bhavya, Chirag, Divya, Warden Sharma, and Dean Rao.

B. The hostel Application (Campus Physical Hierarchy)

Represents university physical housing as a relational hierarchy:
Hostel (e.g., Tagore Boys Hostel) ➔ **Block** (Block A) ➔ **Floor** (1st Floor) ➔ **Room** (Room 101) ➔ **Bed** (Bed 1).
• **Bed Model:** Holds a foreign key to current_occupant. Every bed has an occupancy status (OCCUPIED, VACANT, UNDER_MAINTENANCE).
• **Services:** Provides real-time floor plans, room capacity metrics, and vacancy calculations.

C. The exchange Application & Services Layer

This is the heart of the system, comprising 6 dedicated service modules:

Service Module	Technical Functionality & Algorithms
`Matching Engine` `matching_service.py`	Constructs a **Directed Graph (DiGraph)** where students are nodes and edges represent acceptable room matches. Uses **NetworkX** cycle detection to discover bounded cycles of length 2 to 5. Implements score normalization and rotational deduplication (e.g., [A,B,C] == [B,C,A]).
`Occupancy Engine` `occupancy_service.py`	Guarantees **ACID transactional safety**. When Warden signs off on a swap, this engine sorts Bed Primary Keys in ascending order, acquires row-level locks via select_for_update(), verifies bed occupancy, rotates all occupants atomically, and releases locks.
`Eligibility Service` `eligibility_service.py`	Enforces university bylaws: checks fee dues, disciplinary holds, cooling-off waiting periods between transfers, and gender policy compatibility.
`Proposal Service` `proposal_service.py`	Tracks the lifecycle of multi-party proposals: creates participant voter ballots, tracks unanimous acceptance countdowns, and handles automatic expiration.
`Audit Service` `audit_service.py`	Writes append-only, tamper-proof audit records capturing actor IP, timestamp, previous bed, new bed, and approval authority for administrative transparency.
`Notification Service` `notification_service.py`	Dispatches alerts to student cockpits when proposals are initiated, accepted, rejected, or completed.

5. Advanced Engineering Concepts Explained Simply

A. Graph Cycle Discovery (How Does the System Find Swaps?)

Think of each student as a person holding a key. A directed arrow means: *'I want the key held by that person'*.

• **2-Party Cycle:** Student A wants B's key; Student B wants A's key. (A ➔ B ➔ A)

• **4-Party Rotation:** Student A wants B's key; B wants C's; C wants D's; D wants A's. (A ➔ B ➔ C ➔ D ➔ A)

The system executes a bounded depth-first search (DFS) through NetworkX. To ensure usability, cycles are capped at 5 participants.

B. Concurrency, Race Conditions & Deadlock Immunity

Imagine two proposals are pending at the same moment: Proposal X involves Beds 10 and 20; Proposal Y also involves Bed 20 and Bed 35. If two wardens hit 'Approve' at the exact same millisecond, what happens?

The Risk Without This Engine	How CampusExchange Prevents It
Split / Partial Transfer: Student A moves into Bed 20, but Bed 20's original occupant fails to move into Bed 35, leaving two students in one bed!	**Atomic All-or-Nothing:** Wrapped in transaction.atomic(). Either 100% of students move, or the entire transaction aborts with zero changes.
ABBA Deadlock: Transaction 1 locks Bed 10 and waits for Bed 20. Transaction 2 locks Bed 20 and waits for Bed 10. The database freezes forever!	**Ascending Primary-Key Ordering:** Every transaction sorts Bed IDs (e.g., Bed 10 then Bed 20) before acquiring locks via select_for_update(). Deadlocks are mathematically impossible.

6. User Personas & Role Permissions Matrix

CampusExchange enforces clear separation of authority across university administrative tiers:

Permission / Action	Student	Warden	Chief Warden	Dean / Admin
View Room Cutaway & Allocation	Own Only	Hostel Level	All Hostels	All Hostels
Submit Exchange Request	Yes	No	No	No
Discover & Vote on Proposals	Yes	No	No	No
Review & Approve Same-Hostel Swaps	No	Yes	Yes	Yes
Review Cross-Hostel Swaps	No	No	Yes	Yes
Download Official QR Transfer Pass	Own Pass	Verify Any	Verify Any	Verify Any
Campus Analytics & Audit Logs	No	No	No	Full Access

7. Setup, Testing & Verification Guide

Follow these steps to set up, populate, and test the entire platform on a local machine:

Step 1: Install Dependencies & Run Migrations

pip install -r requirements.txt
python manage.py makemigrations accounts hostel exchange
python manage.py migrate

Step 2: Seed Realistic Campus Demo Data

python manage.py seed_demo
Populates 4 Hostels (Tagore, Sarojini, Aryabhatta, Gargi), 120+ rooms, 240+ beds, demo students (Aarav, Bhavya, Chirag, Divya), wardens, and a ready-to-test 4-party swap cycle (Default password: campus123).

Step 3: Run Dev Server & Switch Personas

python manage.py runserver ➔ Visit http://127.0.0.1:8000/.
Use the top navigation **Persona Switcher** to instantly jump between Student, Warden, and Admin roles.

Step 4: Run Automated Concurrency Suite

python manage.py test tests ➔ Tests multi-threaded atomic row-locking under parallel approval stress.